

Taaskr - Comprehensive 100 Questions for 2+ YOE Java Backend Interview

This document contains EXACTLY 100 unique, highly detailed, multi-paragraph interview questions. It encompasses deep architectural, DevOps, Spring Boot, React, and production troubleshooting scenarios tailored to Taaskr.

Technical Q&A Section

Q1: How does HashMap work internally in Java 8+? How is it used in Taaskr?

Detailed Answer:

Java 8 uses an array of Nodes. When collisions happen, it forms a LinkedList. Once a bucket reaches 8 elements, it converts to a Red-Black Tree. In Taaskr, HashMaps could be used for caching parsed JWT claims or quickly looking up provider availabilities in memory. Furthermore, in a highly concurrent environment like a Spring Boot web server handling thousands of requests, using a standard HashMap can lead to race conditions where the internal linked list forms an infinite loop during a rehash operation. This is why we would explicitly use ConcurrentHashMap for any shared state, which locks only the specific bucket being updated rather than the entire collection.

Interviewer Follow-up: How would concurrent modifications affect a HashMap?

Q2: Explain the difference between interface and abstract class. Why use interfaces for Taaskr Services?

Detailed Answer:

Abstract classes can have state and constructors; interfaces cannot (until Java 8 default methods). In Taaskr, `BookingService` is an interface and `BookingServiceImpl` is the implementation. This decouples the contract from the implementation, making it easier to mock in unit tests and swap implementations. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: Have you used default methods in interfaces?

Q3: What are Java Streams? How would you use them to filter provider availability in Taaskr?

Detailed Answer:

Streams provide a functional approach to processing collections. In Taaskr, you can do `availabilityList.stream().filter(slot -> slot.isAvailable()).collect(Collectors.toList())` to quickly filter available slots without writing explicit loops. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is the difference between `map()` and `flatMap()`?

Q4: How do you handle Exceptions in Java? What is the difference between Checked and Unchecked exceptions?

Detailed Answer:

Checked exceptions (compile-time) must be declared or caught. Unchecked (Runtime) don't. Taaskr uses custom unchecked exceptions like `ResourceNotFoundException` so they bubble up to the `@ControllerAdvice` handler without needing throws declarations everywhere. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: Why does Spring prefer unchecked exceptions?

Q5: What is multithreading? Have you used CompletableFuture in Taaskr?

Detailed Answer:

`CompletableFuture` allows asynchronous non-blocking code. While Taaskr currently processes bookings synchronously, `CompletableFuture` could be used to send email notifications asynchronously after a booking is confirmed, so the user doesn't wait for the SMTP server. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What thread pool does `CompletableFuture` use by default?

Q6: What is the Volatile keyword?

Detailed Answer:

It ensures that updates to a variable are propagated predictably to other threads, preventing them from caching a stale value. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How is this different from `synchronized`?

Q7: Explain the concept of Immutability.

Detailed Answer:

Immutable objects cannot be changed after creation. DTOs in Taaskr (or Java 14+ records) should ideally be immutable to guarantee thread safety when being serialized or passed between layers. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How do you create a truly immutable class in Java?

Q8: What is Garbage Collection and how does it work?

Detailed Answer:

GC automatically frees memory by destroying unreachable objects. Modern JVMs use algorithms like G1GC, which divides the heap into regions and collects the one with the most garbage first. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How do you monitor GC logs in production?

Q9: Difference between String, StringBuilder, and StringBuffer?

Detailed Answer:

String is immutable. `StringBuilder` is mutable and not thread-safe (faster). `StringBuffer` is mutable and thread-safe. Use `StringBuilder` for building dynamic log messages or long SQL queries in Taaskr. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What happens in the string pool when a String is concatenated?

Q10: What are lambda expressions?

Detailed Answer:

Anonymous methods that provide a clear and concise way to implement a functional interface. Used heavily in Taaskr's Stream API filtering. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is a functional interface?

Q11: What is Dependency Injection (DI) and Inversion of Control (IoC)?

Detailed Answer:

IoC delegates control of object creation to a container (Spring). DI is how IoC is implemented (injecting dependencies). In `Taaskr`, `BookingController` doesn't instantiate `BookingService`; Spring injects it, reducing tight coupling. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What are the different types of DI in Spring?

Q12: Why does Taaskr use Constructor Injection over Field Injection (@Autowired)?

Detailed Answer:

Constructor injection allows fields to be `final`, ensuring dependencies are not null at runtime and making classes easier to unit test without Spring's container. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: Can you have circular dependencies with constructor injection?

Q13: Explain the Spring Bean Lifecycle.

Detailed Answer:

Instantiation -> Populate Properties -> `setBeanName` -> `setBeanFactory` -> PreInitialization (BeanPostProcessor) -> `afterPropertiesSet` -> Custom init -> PostInitialization -> Ready -> Destroy. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How can you execute custom logic upon application startup in Taaskr?

Q14: What is @SpringBootApplication?

Detailed Answer:

It's a combination of `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`. It bootstraps the `Taaskr` application. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How can you exclude an auto-configuration class?

Q15: What is @ControllerAdvice and how is it used in Taaskr?

Detailed Answer:

It handles global exceptions. `GlobalExceptionHandler` uses it to catch `ResourceNotFoundException` and return a standardized JSON error response with appropriate HTTP status codes (e.g., 404). In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How does @ExceptionHandler work?

Q16: Explain @ConfigurationProperties vs @Value.

Detailed Answer:

Both inject properties. `@Value` is used for single fields (like `#{jwt.secret}`). `@ConfigurationProperties` binds a prefix of properties to a POJO, making it strongly typed and easier to validate (e.g., Razorpay properties). A critical vulnerability with JWTs is token revocation. Since they are stateless, you cannot simply delete a session from a database to log a user out. If a token is stolen, it remains valid until expiration. To mitigate this in a production system, we would implement a Redis blacklist where logged-out tokens are stored until their natural expiration time, and the `JwtAuthenticationFilter` would check Redis on every request.

Interviewer Follow-up: How do you validate @ConfigurationProperties?

Q17: What is Spring Boot Actuator?

Detailed Answer:

It provides production-ready features like health checks, metrics, and environment info. We can monitor `Taaskr`'s health via `/actuator/health`. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How do you secure Actuator endpoints?

Q18: How does Spring Boot Embedded Tomcat work?

Detailed Answer:

Instead of deploying a WAR file to a standalone Tomcat, Spring Boot packages Tomcat as a dependency inside the JAR, starting the server programmatically. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How would you change the embedded server to Undertow?

Q19: What are Spring Profiles?

Detailed Answer:

They allow conditional bean creation and property loading based on the environment. `Taaskr` uses `application-prod.properties` for production and `application.properties` for local dev. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How do you activate a specific profile via command line?

Q20: How does @Transactional work in Taaskr?

Detailed Answer:

It ensures that a series of DB operations either all commit or all rollback. It uses AOP proxies around the Service method. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What happens if a @Transactional method catches an exception internally?

Q21: What is the difference between @Controller and @RestController?

Detailed Answer:

`@RestController` is `@Controller` + `@ResponseBody`. It automatically serializes return objects to JSON. `Taaskr` uses it for all API endpoints to serve the React frontend. On the frontend, managing state efficiently is crucial. While `Taaskr` might use React Context or local state for simple data, scaling the application would require a dedicated state manager like Redux or Zustand. Furthermore, optimizing re-renders using `useMemo` and `useCallback` becomes necessary when dealing with complex booking tables or maps to prevent the UI from freezing during rapid user interactions.

Interviewer Follow-up: How is the JSON serialization handled under the hood? (Jackson)

Q22: Explain the HTTP status codes used in Taaskr.

Detailed Answer:

200 OK (Success), 201 Created (New booking), 400 Bad Request (Validation failure), 401 Unauthorized (Invalid JWT), 403 Forbidden (Customer accessing admin route), 404 Not Found (Provider not found), 500 Internal Error (DB down). A critical vulnerability with JWTs is token revocation. Since they are stateless, you cannot simply delete a session from a database to log a user out. If a token is stolen, it remains valid until expiration. To mitigate this in a production system, we would implement a Redis blacklist where logged-out tokens are stored until their natural expiration time, and the `JwtAuthenticationFilter` would check Redis on every request.

Interviewer Follow-up: What is the difference between 401 and 403?

Q23: What is idempotency in REST?

Detailed Answer:

Making the same request multiple times has the same effect as making it once. GET, PUT, DELETE are idempotent. POST is not. Taaskr's POST to `/api/bookings` creates a new booking every time. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How would you design a POST to be idempotent? (Idempotency key)

Q24: How does `@Valid` work for DTO validation?

Detailed Answer:

It triggers Bean Validation (Hibernate Validator) on the incoming DTO. If constraints like `@NotNull` or `@Email` on `RegisterRequest` fail, it throws `MethodArgumentNotValidException`. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How do you customize the error message returned to the user?

Q25: How does CORS work and how is it configured in Taaskr?

Detailed Answer:

Cross-Origin Resource Sharing allows the React frontend on port 5173 to call Spring Boot on 8081. In Taaskr, it's configured globally in `SecurityConfig`. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What is a CORS preflight request?

Q26: What is the N+1 problem in JPA? How would you solve it in Taaskr?

Detailed Answer:

It occurs when JPA executes 1 query to fetch N parent entities (e.g., Providers), and then N additional queries to fetch their lazy-loaded children (e.g., Services). Solve it using `JOIN FETCH` in JPQL or `@EntityGraph`. Additionally, relying entirely on Spring Data JPA for complex reporting queries can lead to memory exhaustion and severe N+1 issues. For complex analytical dashboards, it is often better to use `JdbcTemplate`, native SQL queries, or a tool like `JOOQ`, because they bypass the Hibernate persistence context entirely, avoiding the overhead of dirty checking and object-relational impedance mismatch.

Interviewer Follow-up: When should you use LAZY vs EAGER fetching?

Q27: Explain `@OneToMany` and `@ManyToOne` relationships.

Detailed Answer:

`@OneToMany` implies a parent has many children (e.g., one User has many Bookings). `@ManyToOne` is the inverse. In Taaskr, a Booking has a `@ManyToOne` relationship to the ProviderProfile. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is `mappedBy`?

Q28: What is the first-level cache in Hibernate?

Detailed Answer:

It's tied to the `JPA EntityManager` (transaction). If you fetch the same Booking twice in one transaction, the DB is hit once. Taaskr uses this implicitly during complex updates. Additionally, relying entirely on Spring Data JPA for complex reporting queries can lead to memory exhaustion and severe N+1 issues. For complex analytical dashboards, it is often better to use `JdbcTemplate`, native SQL queries, or a tool like `JOOQ`, because they bypass the Hibernate persistence context entirely, avoiding the overhead of dirty checking and object-relational impedance mismatch.

Interviewer Follow-up: What is the second-level cache?

Q29: What is Dirty Checking in Hibernate?

Detailed Answer:

Hibernate tracks changes to managed entities. At the end of a transaction, it automatically generates SQL UPDATES for modified entities without explicit `save()` calls. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How can you bypass dirty checking for read-only transactions?

Q30: What is the difference between `save()` and `saveAndFlush()` in Spring Data JPA?

Detailed Answer:

`save()` queues the operation until transaction commit. `saveAndFlush()` immediately sends the SQL to the DB, useful if subsequent logic depends on DB triggers or constraints. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: Why would you ever need `saveAndFlush`?

Q31: How would you handle DB migrations in Taaskr?

Detailed Answer:

Currently using `ddl-auto=update`. In production, this is dangerous. We should use Flyway or Liquibase to manage versioned SQL scripts. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What happens if `ddl-auto=create-drop` is used in production?

Q32: What are DB Indexes? Where would you add them in Taaskr?

Detailed Answer:

Indexes speed up data retrieval. I would index the `email` column in `User` since it's frequently queried during login, and the `status` column in `Booking` for filtering. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What are the downsides of too many indexes?

Q33: What is Connection Pooling?**Detailed Answer:**

Reusing DB connections instead of opening a new one per request. `Taaskr` uses `HikariCP` (Spring Boot default) to maintain a pool, reducing latency and DB load. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How do you configure the pool size?

Q34: How does JWT Authentication work in Taaskr?**Detailed Answer:**

Upon login, server validates credentials and generates a signed JWT. The React app sends it in the `Authorization: Bearer <token>` header. `JwtAuthenticationFilter` validates the signature and populates `SecurityContext`. A critical vulnerability with JWTs is token revocation. Since they are stateless, you cannot simply delete a session from a database to log a user out. If a token is stolen, it remains valid until expiration. To mitigate this in a production system, we would implement a Redis blacklist where logged-out tokens are stored until their natural expiration time, and the `JwtAuthenticationFilter` would check Redis on every request.

Interviewer Follow-up: What happens if the JWT is stolen?

Q35: Why is JWT stateless?**Detailed Answer:**

The token contains all necessary user claims and the signature. The server doesn't need to store a session ID in memory or DB to verify the user. A critical vulnerability with JWTs is token revocation. Since they are stateless, you cannot simply delete a session from a database to log a user out. If a token is stolen, it remains valid until expiration. To mitigate this in a production system, we would implement a Redis blacklist where logged-out tokens are stored until their natural expiration time, and the `JwtAuthenticationFilter` would check Redis on every request.

Interviewer Follow-up: How do you invalidate a JWT before it expires?

Q36: What is the SecurityFilterChain in Spring Security?**Detailed Answer:**

It configures which endpoints are public (e.g., `/api/auth/**`) and which require authentication. It also registers custom filters like the JWT filter. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How does `OncePerRequestFilter` differ from a regular `Filter`?

Q37: What is PasswordEncoder and why use BCrypt?**Detailed Answer:**

Passwords must be hashed, not stored in plaintext. `BCrypt` includes a salt to protect against rainbow table attacks and is intentionally slow to deter brute-forcing. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How does `BCrypt` verify a password if the salt is random?

Q38: What is the difference between Authentication and Authorization?**Detailed Answer:**

Authentication proves WHO you are (JWT signature). Authorization determines WHAT you can do (Role = ADMIN or PROVIDER). A critical vulnerability with JWTs is token revocation. Since they are stateless, you cannot simply delete a session from a database to log a user out. If a token is stolen, it remains valid until expiration. To mitigate this in a production system, we would implement a Redis blacklist where logged-out tokens are stored until their natural expiration time, and the `JwtAuthenticationFilter` would check Redis on every request.

Interviewer Follow-up: How do you secure a specific method with roles in Spring?

Q39: How would you scale Taaskr from 1,000 to 1,000,000 users?**Detailed Answer:**

Move away from single EC2. Add an Application Load Balancer. Run Spring Boot in stateless ECS/EKS containers. Scale the Aiven MySQL via read replicas. Add Redis for caching the provider catalog. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How would you handle user sessions in a scaled environment? (JWT handles this naturally)

Q40: If the DB is overwhelmed with reads, what caching strategy would you use?**Detailed Answer:**

Introduce Redis. Cache the Provider Catalog and Service lists using `@Cacheable`, since this data doesn't change every second. Invalidate cache on updates. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is Cache Stampede?

Q41: How would you introduce asynchronous processing in Taaskr?**Detailed Answer:**

Use Kafka or RabbitMQ. When a booking is created, instead of waiting for payment/email confirmation inline, publish an event to a Kafka topic. A separate consumer microservice handles the email/payment sync. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: Why not just use `@Async`?

Q42: What is Rate Limiting and how would you implement it?**Detailed Answer:**

Preventing abuse by limiting requests per IP. Can be implemented via an API Gateway (Kong/AWS API Gateway) or a `Bucket4j` filter using Redis to track limits across distributed nodes. When deploying to AWS, relying on static EC2 instances is often an anti-pattern for modern applications. The ideal progression for `Taaskr` would be to dockerize the Spring Boot application and deploy it via ECS (Elastic Container Service) or EKS (Kubernetes) with an Auto Scaling Group. This allows the system to automatically spin up new instances during traffic spikes and scale down during off-hours, saving costs and ensuring high availability.

Interviewer Follow-up: What HTTP status code is returned when limit is exceeded? (429)

Q43: Explain Monolith vs Microservices for Taaskr.

Detailed Answer:

Taaskr is a monolith. It's easy to deploy and test. As the team and traffic grow, we could split it into `User-Service`, `Booking-Service`, and `Payment-Service`. While microservices solve scaling and organizational issues, they introduce massive complexity in observability. You can no longer just look at a single log file. You need distributed tracing (like `OpenTelemetry`) where a unique `Trace ID` is injected at the API Gateway and passed through HTTP headers to every downstream service. This allows you to visualize the exact path and latency of a request across the entire distributed system.

Interviewer Follow-up: What is a major challenge when splitting into microservices? (Distributed transactions)

Q44: Why did you use Nginx in front of Spring Boot?

Detailed Answer:

Nginx acts as a reverse proxy, handling SSL termination, serving the static React files extremely efficiently, and proxying only API requests to Spring Boot, hiding port 8081 from the outside. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What does `try_files $uri /index.html do?`

Q45: How do you deploy the Taaskr application?

Detailed Answer:

Run `npm run build` for React. Run `mvn clean package` for Spring Boot. Transfer JAR and `dist` to EC2. Configure Nginx to serve `dist` and proxy `/api` to the Java JAR running via `systemd`. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: Why use `systemd` to run the JAR instead of `nohup java -jar?`

Q46: What is the significance of Environment Variables in Taaskr?

Detailed Answer:

We inject `DB_HOST`, `JWT_SECRET`, and `RAZORPAY_SECRET` via env vars so sensitive data isn't hardcoded in `application.properties` or committed to Git. A critical vulnerability with JWTs is token revocation. Since they are stateless, you cannot simply delete a session from a database to log a user out. If a token is stolen, it remains valid until expiration. To mitigate this in a production system, we would implement a Redis blacklist where logged-out tokens are stored until their natural expiration time, and the `JwtAuthenticationFilter` would check Redis on every request.

Interviewer Follow-up: How did you verify variables were loaded on the Linux server? (`printenv`, `/proc/<pid>/environ`)

Q47: What happens when DNS fails (NXDOMAIN) for the Aiven database?

Detailed Answer:

Spring Boot will fail to start and throw `UnknownHostException`. Fixed by ensuring the EC2 instance can reach the public internet and resolve DNS. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How do you debug DNS on Linux? (`nslookup`, `dig`)

Q48: How does Docker simplify deployment?

Detailed Answer:

It packages the app and its dependencies (JRE 17) into a standard image. It eliminates the 'works on my machine' problem. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is a multi-stage Docker build?

Q49: How did you troubleshoot a 502 Bad Gateway in Nginx?

Detailed Answer:

Checked `Nginx error.log`. It usually means Nginx can't connect to port 8081. Checked if Spring Boot was running via `systemctl status orps -ef | grep java`. Found out Java process died due to OOM. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How do you allocate more memory to the JVM?

Q50: How do you identify slow database queries in Taaskr?

Detailed Answer:

Enable Spring Boot slow query logging, or use DB monitoring tools in Aiven. We can also use APM tools like New Relic or Datadog. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What is `EXPLAIN` in MySQL?

Q51: What is central logging and why is it important?

Detailed Answer:

In a scaled environment with multiple EC2 instances, viewing logs locally is impossible. We send logs to an ELK stack (Elasticsearch, Logstash, Kibana) to search across all instances. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How do you add correlation IDs to logs? (MDC in Spring)

Q52: How is Razorpay integrated into Taaskr?

Detailed Answer:

Server calls Razorpay API to create an Order and sends the `OrderID` to React. React opens the checkout widget. Upon success, React sends Payment ID and Signature to the server. Server verifies the HMAC SHA256 signature. On the frontend, managing state efficiently is crucial. While Taaskr might use `React Context` or local state for simple data, scaling the application would require a dedicated state manager like `Redux` or `Zustand`. Furthermore, optimizing re-renders using `useMemo` and `useCallback` becomes necessary when dealing with complex booking tables or maps to prevent the UI from freezing during rapid user interactions.

Interviewer Follow-up: Why must the signature be verified on the backend?

Q53: What happens if the Razorpay webhook fails?

Detailed Answer:

If we rely on webhooks, the payment status might remain 'PENDING'. A cron job should periodically check pending payments against the Razorpay API. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is a webhook?

Q54: What design patterns are used in Taaskr?

Detailed Answer:

1. **Dependency Injection:** Used everywhere via Spring.\n2. **Factory/Builder Pattern:** Used in creating DTOs or JWT generation.\n3. **Singleton:** Spring beans are singletons by default.\n4. **Repository Pattern:** Spring Data JPA abstracts DB operations. Under the hood, Spring manages these beans in the ApplicationContext. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: Is a Spring Singleton thread-safe?

Q55: What is the difference between git merge and git rebase?

Detailed Answer:

Merge creates a new commit combining the histories. Rebase rewrites history by appending your branch commits on top of the target branch, keeping history linear. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: When should you NOT use rebase? (On shared public branches)

Q56: How do you remove a secret accidentally committed to Git?

Detailed Answer:

Using `git filter-repo` or `BFG Repo-Cleaner` to rewrite history and remove the file completely, then forcefully pushing. Also, rotate the secret immediately. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What does `git stash` do?

Q57: How would you implement a distributed lock in Taaskr to prevent double bookings?

Detailed Answer:

Use Redis with Redisson or a dedicated lock table in MySQL to ensure only one node can execute the booking transaction. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is the redlock algorithm?

Q58: Explain the proxy pattern used in Spring AOP.

Detailed Answer:

Spring creates dynamic proxies around beans to inject cross-cutting concerns like `@Transactional` or `@PreAuthorize`. Under the hood, Spring manages these beans in the ApplicationContext. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What happens if a method calls another `@Transactional` method in the same class?

Q59: How do you handle migrations and backwards compatibility for REST APIs?

Detailed Answer:

Use API versioning (e.g., `/api/v1/bookings`). Ensure DB migrations (flyway) don't drop columns immediately. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: Header vs URL versioning?

Q60: How would you implement pagination for the Provider Catalog?

Detailed Answer:

Use Spring Data's `Pageable` interface in the repository and pass page/size parameters in the controller. Under the hood, Spring manages these beans in the ApplicationContext. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: Why is pagination better than returning the whole list?

Q61: Describe the exact process of debugging a 'Connection Refused' error on production.

Detailed Answer:

1. Verify DB is running. 2. Ping DB host. 3. Check security groups/firewalls blocking port 3306. 4. Verify DB credentials in env vars. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What command checks open ports locally? (`netstat` or `ss`)

Q62: What is the purpose of the `@Enumerated(EnumType.STRING)` annotation?

Detailed Answer:

It maps the Java Enum to a VARCHAR column in MySQL. Default is `ORDINAL` (integer), which breaks if the enum order changes. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What happens to old records if an enum value is removed?

Q63: How does React Router handle client-side routing and how does Nginx support it?

Detailed Answer:

React Router manipulates the browser history API. Nginx uses `try_files $uri /index.html` to serve the React index page for deep links, letting React handle the route. Beyond just routing, Nginx acts as a powerful buffer. Without Nginx, a slow client (like a mobile user on a 3G network) could tie up a Tomcat thread in Spring Boot for several seconds just reading the response. Nginx reads the entire response from Tomcat almost instantly, frees up the Java thread, and then slowly streams the data to the client. This drastically increases the throughput of the Java application.

Interviewer Follow-up: What happens if `try_files` is missing?

Q64: How do you manage application memory limits in a Docker container?

Detailed Answer:

Pass JVM flags like `-XX:MaxRAMPercentage=75.0` so Java respects container limits instead of physical host memory. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What happens if a container exceeds memory? (`OOMKilled`)

Q65: How do you test the repository layer in Spring Boot?

Detailed Answer:

Use `@DataJpaTest`, which configures an in-memory database (H2) and only loads JPA beans for fast testing. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How do you rollback transactions in tests? (Automatic in `@DataJpaTest`)

Q66: Explain CI/CD in the context of Taaskr.

Detailed Answer:

Continuous Integration (GitHub Actions) runs `mvn test` on PRs. Continuous Deployment automatically builds the JAR/Docker image and deploys to EC2. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is a blue/green deployment?

Q67: Why should API responses wrap data in a standard object (e.g., `ApiResponse`)?

Detailed Answer:

It provides a consistent contract for the frontend, always including status, message, and payload, making generic error handling easier in Axios. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How do you implement this globally in Spring?

Q68: How do you handle timezones in Taaskr?

Detailed Answer:

Store all dates in UTC in MySQL. The frontend (React) converts UTC to the user's local timezone for display. On the frontend, managing state efficiently is crucial. While Taaskr might use React Context or local state for simple data, scaling the application would require a dedicated state manager like Redux or Zustand. Furthermore, optimizing re-renders using `useMemo` and `useCallback` becomes necessary when dealing with complex booking tables or maps to prevent the UI from freezing during rapid user interactions.

Interviewer Follow-up: What Java 8 classes handle timezones? (`ZonedDateTime`)

Q69: What is the CAP theorem?

Detailed Answer:

Consistency, Availability, Partition Tolerance. A distributed system can only guarantee two. Taaskr's relational DB prioritizes Consistency over Availability in the event of a partition. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: Does NoSQL prioritize differently?

Q70: How would you optimize the React bundle size?

Detailed Answer:

Use code splitting with `React.lazy()` for distinct routes (e.g., Admin Dashboard vs Customer Dashboard) to avoid loading everything on initial hit. On the frontend, managing state efficiently is crucial. While Taaskr might use React Context or local state for simple data, scaling the application would require a dedicated state manager like Redux or Zustand. Furthermore, optimizing re-renders using `useMemo` and `useCallback` becomes necessary when dealing with complex booking tables or maps to prevent the UI from freezing during rapid user interactions.

Interviewer Follow-up: What tool analyzes bundle size?

Q71: What is the significance of Vite over Create React App?

Detailed Answer:

Vite uses native ES modules during development, resulting in near-instant hot module replacement (HMR) and significantly faster build times. On the frontend, managing state efficiently is crucial. While Taaskr might use React Context or local state for simple data, scaling the application would require a dedicated state manager like Redux or Zustand. Furthermore, optimizing re-renders using `useMemo` and `useCallback` becomes necessary when dealing with complex booking tables or maps to prevent the UI from freezing during rapid user interactions.

Interviewer Follow-up: What bundler does Vite use for production? (Rollup)

Q72: How do you prevent XSS attacks in Taaskr?

Detailed Answer:

React automatically escapes HTML to prevent XSS. On the backend, we validate input and can use a library to sanitize dangerous HTML tags if users input rich text. On the frontend, managing state efficiently is crucial. While Taaskr might use React Context or local state for simple data, scaling the application would require a dedicated state manager like Redux or Zustand. Furthermore, optimizing re-renders using `useMemo` and `useCallback` becomes necessary when dealing with complex booking tables or maps to prevent the UI from freezing during rapid user interactions.

Interviewer Follow-up: What is a stored XSS attack?

Q73: What is CSRF and why does JWT prevent it?

Detailed Answer:

Cross-Site Request Forgery. JWTs stored in `localStorage` and sent via Bearer headers aren't automatically sent by the browser like cookies, mitigating CSRF. A critical vulnerability with JWTs is token revocation. Since they are stateless, you cannot simply delete a session from a database to log a user out. If a token is stolen, it remains valid until expiration. To mitigate this in a production system, we would implement a Redis blacklist where logged-out tokens are stored until their natural expiration time, and the `JwtAuthenticationFilter` would check Redis on every request.

Interviewer Follow-up: What if the JWT is stored in an `HttpOnly` cookie?

Q74: How do you ensure secure DB connections?

Detailed Answer:

The Aiven URL specifies `sslMode=REQUIRED`. This encrypts data in transit between the EC2 instance and the Aiven servers. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How do you manage SSL certs for Nginx? (Certbot/Let's Encrypt)

Q75: Explain the Saga pattern.

Detailed Answer:

In microservices, transactions span multiple services. Saga executes a sequence of local transactions. If one fails, it executes compensating transactions to rollback the previous ones. While microservices solve scaling and organizational issues, they introduce massive complexity in observability. You can no longer just look at a single log file. You need distributed tracing (like OpenTelemetry) where a unique Trace ID is injected at the API Gateway and passed through HTTP headers to every downstream service. This allows you to visualize the exact path and latency of a request across the entire distributed system.

Interviewer Follow-up: How is this different from 2PC?

Q76: What is API Gateway?

Detailed Answer:

A single entry point for all clients. It handles routing, rate limiting, authentication, and SSL termination. Kong or Spring Cloud Gateway. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: How does it differ from a Reverse Proxy?

Q77: How do you implement Health Checks?**Detailed Answer:**

Spring Boot Actuator exposes `/actuator/health`. An external monitoring service or Load Balancer hits this endpoint to verify the node is alive. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What makes a health check fail? (DB down, Disk full)

Q78: Explain the concept of Circuit Breaker.**Detailed Answer:**

If a downstream service (like Razorpay) is failing, the circuit breaker trips and immediately returns an error or fallback, preventing cascading failures and thread exhaustion. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What library implements this? (Resilience4j)

Q79: What is a Thread Dump?**Detailed Answer:**

A snapshot of the state of all threads. Used to diagnose deadlocks or high CPU usage. Captured via `jstack`. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What is a Heap Dump?

Q80: How do you prevent duplicate user registrations?**Detailed Answer:**

Add a `UNIQUE` constraint on the `email` column in MySQL and handle the `DataIntegrityViolationException` in Spring Boot to return a 400 Bad Request. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: Should this logic also exist in the Service layer?

Q81: What is the difference between @Component, @Service, and @Repository?**Detailed Answer:**

They all register beans. `@Service` is a semantic marker for business logic. `@Repository` automatically translates SQL exceptions into Spring's `DataAccessException` hierarchy. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What does `@Component` do?

Q82: How do you handle configuration secrets in AWS?**Detailed Answer:**

Instead of env vars on EC2, use AWS Secrets Manager or Parameter Store. Fetch them at startup via Spring Cloud AWS. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: Why is this better than env vars?

Q83: What is the Builder Pattern?**Detailed Answer:**

It separates the construction of a complex object from its representation. Helpful for DTOs with many optional fields. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: How do you use it in Java? (@Builder via Lombok)

Q84: How does `npm ci` differ from `npm install`?**Detailed Answer:**

`npm ci` strictly installs exactly what is in `package-lock.json` and deletes `node_modules` first, ensuring reproducible builds. `npm install` might update the lock file. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: When should you use `npm install`?

Q85: What happens if the JVM runs out of memory?**Detailed Answer:**

It throws `OutOfMemoryError`. The process might hang. We should configure `-XX:+HeapDumpOnOutOfMemoryError` to analyze the heap dump. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What tool reads a heap dump? (Eclipse MAT)

Q86: Explain the difference between `Long` and `long` in Java Entities.**Detailed Answer:**

`Long` is an object wrapper and can be null. `long` is a primitive and defaults to 0. IDs should be `Long` so JPA knows when an entity is unsaved (null ID). Additionally, relying entirely on Spring Data JPA for complex reporting queries can lead to memory exhaustion and severe N+1 issues. For complex analytical dashboards, it is often better to use JdbcTemplate, native SQL queries, or a tool like jOOQ, because they bypass the Hibernate persistence context entirely, avoiding the overhead of dirty checking and object-relational impedance mismatch.

Interviewer Follow-up: What is autoboxing?

Q87: How do you trace a request across multiple microservices?

Detailed Answer:

Use Distributed Tracing (e.g., Sleuth/Zipkin or OpenTelemetry). A unique Trace ID is generated at the Gateway and passed via HTTP headers. While microservices solve scaling and organizational issues, they introduce massive complexity in observability. You can no longer just look at a single log file. You need distributed tracing (like OpenTelemetry) where a unique Trace ID is injected at the API Gateway and passed through HTTP headers to every downstream service. This allows you to visualize the exact path and latency of a request across the entire distributed system.

Interviewer Follow-up: What is the difference between Trace ID and Span ID?

Q88: What is a materialized view in databases?

Detailed Answer:

A database object that contains the results of a query. Unlike a normal view, it stores the data on disk, speeding up complex aggregations at the cost of stale data. Monitoring database performance goes beyond just slow query logs. A 2 YOE developer should understand how to use tools like `EXPLAIN ANALYZE` to read query execution plans. Missing indexes often cause full table scans. However, over-indexing slows down `INSERT` and `UPDATE` operations because the database must update the B-Tree for every index. Balancing read vs write performance is key in a system like Taaskr where bookings are constantly updated.

Interviewer Follow-up: How do you refresh it?

Q89: How does Spring Data JPA generate queries from method names?

Detailed Answer:

It parses method names like `findByEmailAndStatus()` into JPQL criteria queries at runtime. Under the hood, Spring manages these beans in the `ApplicationContext`. By default, beans are Singletons, which means they must be stateless. If you inject a stateful component into a singleton service, you will encounter severe thread-safety issues because multiple HTTP threads will mutate the same instance variables simultaneously. This is a critical concept for 2 YOE developers to master.

Interviewer Follow-up: What happens if the method name has a typo? (Fails on startup)

Q90: Explain the purpose of `flyway_schema_history` table.

Detailed Answer:

It tracks which migration scripts (e.g., `V1__init.sql`) have been executed so Flyway doesn't run them again on startup. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: What happens if a checksum changes?

Q91: What is the difference between `@Mock` and `@InjectMocks` in Mockito?

Detailed Answer:

`@Mock` creates a mock object. `@InjectMocks` creates an instance of the class being tested and injects the created `@Mocks` into it. In a production environment, this requires careful monitoring and alerting setup to ensure high availability and prevent resource exhaustion. Developers must write defensive code, assuming network partitions and downstream failures will inevitably occur.

Interviewer Follow-up: When do you use `@MockBean`? (Spring Boot integration tests)

Q92: How do you handle Distributed Transactions if Taaskr moves to microservices?

Detailed Answer:

In a microservices architecture, a single transaction (like creating a booking and charging a payment) might span multiple independent databases. We cannot use standard `@Transactional` because there is no single database connection. Instead, we must use the Saga Pattern. We break the transaction into a series of local transactions. If one step fails (e.g., payment fails), the Saga orchestrator publishes compensating events to rollback the previous steps (e.g., cancel the booking). This guarantees eventual consistency without holding long-lived locks.

Interviewer Follow-up: What is the Two-Phase Commit (2PC) protocol and why is it generally avoided in modern cloud architectures?

Q93: Explain the concept of Database Connection Pool Exhaustion in Taaskr.

Detailed Answer:

Taaskr uses HikariCP for connection pooling. If the pool size is 10, and 10 concurrent requests execute slow database queries (e.g., waiting for a 3rd party API inside a `@Transactional` block), the pool becomes exhausted. The 11th request will block waiting for a connection, and eventually throw a `SQLTransientConnectionException`. To fix this, NEVER make slow network calls (like Razorpay API) inside a database transaction. Fetch data, close the transaction, make the API call, then open a new transaction to save the result.

Interviewer Follow-up: How can you monitor HikariCP metrics in Spring Boot?

Q94: How would you implement secure password resets in Taaskr?

Detailed Answer:

When a user requests a reset, the server generates a cryptographically secure random token (e.g., using `SecureRandom`), hashes it, stores the hash in the database with an expiration time (e.g., 15 minutes), and emails the plain token link to the user. When the user clicks the link, the server hashes the provided token, compares it to the database, and if valid, allows the password change. We hash the token in the DB to prevent an attacker with read-only DB access from hijacking accounts.

Interviewer Follow-up: Why should the token expire quickly?

Q95: What is an API Gateway, and how would it benefit Taaskr as it scales?

Detailed Answer:

An API Gateway (like Spring Cloud Gateway or Kong) sits between the Nginx proxy and the backend services. It acts as a centralized enforcement point. It can handle JWT validation, rate limiting (preventing DDoS), request routing, and SSL termination. This offloads these cross-cutting concerns from the Spring Boot application, allowing the backend services to focus purely on business logic.

Interviewer Follow-up: How does rate limiting work at the API Gateway level? (e.g., Token Bucket algorithm).

Q96: How do you ensure zero-downtime deployments for Taaskr?

Detailed Answer:

Currently, restarting the systemd service causes a few seconds of downtime. To achieve zero-downtime, we use a Blue-Green or Rolling deployment. In Blue-Green, we spin up a completely new instance of the app (Green) while the old one (Blue) serves traffic. Once Green is healthy (verified via `/actuator/health`), we flip the Load Balancer or Nginx configuration to point to Green, and then terminate Blue. This ensures users never see a 502 Bad Gateway.

Interviewer Follow-up: How do database schema changes complicate Blue-Green deployments?

Q97: What are the advantages of using Vite over Create React App (CRA) in Taaskr's frontend?

Detailed Answer:

CRA uses Webpack, which bundles the entire application before it can be served during development. As the app grows, this takes seconds or even minutes. Vite uses native ES Modules in the browser. It only compiles the exact file you are currently editing, resulting in near-instant Hot Module Replacement (HMR) regardless of app size. For production, Vite uses Rollup, producing highly optimized static bundles.

Interviewer Follow-up: What is the difference between ES Modules and CommonJS?

Q98: Explain how you would monitor the JVM heap memory in production.

Detailed Answer:

We would use Spring Boot Actuator coupled with Micrometer to expose JVM metrics in Prometheus format. A Prometheus server scrapes these metrics, and Grafana visualizes them. We would set up alerts on metrics like `jvm.memory.used` and `jvm.gc.pause`. If the Old Gen space is constantly full and GC pauses exceed a few seconds, it indicates a memory leak, prompting us to take a heap dump (`jmap`) for analysis in Eclipse Memory Analyzer.

Interviewer Follow-up: What is a memory leak in Java if the Garbage Collector is supposed to handle memory automatically?

Q99: How does Taaskr's @ControllerAdvice handle varying types of exceptions globally?

Detailed Answer:

The `@ControllerAdvice` class contains methods annotated with `@ExceptionHandler`. When any controller throws an exception, Spring intercepts it and routes it to the matching handler. For example, a `ResourceNotFoundException` is routed to a handler that returns a 404 status and a standardized `ApiError` DTO. A `MethodArgumentNotValidException` (from failed `@Valid` constraints) is routed to a handler that returns a 400 status and a list of specific field validation errors. This ensures the React frontend always receives a predictable error format.

Interviewer Follow-up: How do you catch unhandled `RuntimeExceptions` without leaking stack traces to the client?

Q100: Explain the significance of configuring H2 database for testing instead of using the production MySQL database.

Detailed Answer:

Testing against a real MySQL database often leads to test flakiness due to lingering state and slower execution times. By configuring H2 (an in-memory database) in `pplication-test.properties`, we ensure that every test suite spins up a fresh, isolated database in milliseconds. This is crucial for unit tests and `@DataJpaTest` slices, ensuring that tests are fast, deterministic, and independent of external infrastructure. However, for full end-to-end integration tests, we might use `Testcontainers` to spin up a real MySQL Docker instance to verify database-specific dialect behaviors that H2 might miss.

Interviewer Follow-up: What is `Testcontainers` and why is it superior to H2 for integration testing?
